

PROGRAMACIÓN

Programador Universitario - Licenciatura en Informática - Ingeniería en Informática
Facultad de Ciencias Exactas y Tecnología - UNT

Trabajo Práctico N° 5

TEMA: Arreglos

“Un arreglo es una colección de variables ordenadas e indexadas, todas de idéntico tipo que se referencian usando un nombre común”.

Funciones de biblioteca del archivo de cabecera <string.h>

Las variables cad1 y cad2 son arreglos de caracteres.

int strlen(cad1)	Retorna la longitud de cad1	
int strcmp(cad1, cad2)	Compara cad1 con cad2, carácter a carácter	SI (cad1[i] < cad2 [i]) ENTONCES Retorna entero< 0 SI (cad1[i] = cad2 [i]) ENTONCES Retorna 0 SI (cad1[i] > cad2 [i]) ENTONCES Retorna entero> 0
int strncmp(cad1,cad2,n)	Compara hasta n caracteres de la cad1 con cad2 sin diferenciar mayúsculas de minúscula.	SI (cad1[i] < cad2 [i]) ENTONCES Retorna entero < 0 SI (cad1[i] = cad2 [i]) ENTONCES Retorna 0 SI (cad1[i] > cad2 [i]) ENTONCES Retorna entero > 0
char *strcpy(cad1, cad2)	Copia cad2 a cad1, incluyendo el terminador “\0”. Retorna cad1	
char *strncpy(cad1, cad2,n)	Copia hasta n caracteres de la cad2 a cad1. Retorna cad1. Rellena con “\0” si cad2 tiene menos de n caracteres	
char *strcat(cad1, cad2)	Concatena la cad2 al final de cad1. Retorna cad1	

Números aleatorios:

```
#include <stdlib.h> // Incluye las funciones rand() y srand()
#include <time.h> // Incluye las funciones time(). Time(null) devuelve el tiempo actual.
srand(time(NULL)); // Inicializar la semilla con la hora actual. Genera distintos aleatorios
                    en cada ejecución.
rand()             // Genera aleatorio de 0 a 32767
rand() % 101;      // Genera aleatorio de 0 a 100
inicio + rand() % (fin - inicio + 1); // Fórmula general para un rango [inicio,fin]
```



Diseñar y escribir un algoritmo al menos para el punto 1. Codificar en Lenguaje C todos los puntos y realizar pruebas para distintos conjuntos de datos, si son datos aleatorios ejecutar el programa varias veces..

PROGRAMACIÓN

Programador Universitario - Licenciatura en Informática - Ingeniería en Informática
Facultad de Ciencias Exactas y Tecnología - UNT

1. Todos contra todos

En la etapa de “**Los Rounds**” del programa *La Voz Argentina 2025*, cada coach dedica un episodio a su equipo. Durante ese episodio:

1. **Presentaciones individuales:** Los **8 participantes** del equipo Miranda cantan en solitario.
2. **Puntajes secretos:** Los **3 coaches restantes** (no el líder del equipo) otorgan a cada participante un puntaje secreto entre **1 y 10**.
3. **Salvados por el coach:** Finalizadas todas las presentaciones, el **coach líder** salva a **3 participantes**, a su criterio.
4. **Eliminación automática:** Entre los participantes no salvados, el que tenga la **menor puntuación total** es eliminado del certamen.

Requisitos

Simular este proceso en **C**, utilizando un **arreglo** para almacenar la información:

- Generar aleatoriamente las puntuaciones de los coaches para cada participante (valores entre **1 y 10**).
- Calcular el **puntaje total** de cada participante y guardarlo en un arreglo, donde cada posición representa a un participante.

Participante 1: 15 ptos.

Participante 2: 21 ptos.

...

Participante 8: 17 ptos.

- Los **3 participantes salvados** por el coach líder deben ser ingresados como datos por teclado.
 - Los números ingresados deben estar comprendidos en el rango [1,8].
 - Que no se repita el mismo participante más de una vez.
 - De no pasar la validación, se debe indicar el motivo y pedir nuevamente los datos.
- Entre los **restantes**, determinar cuál es el **eliminado** (el de menor puntaje).
- La puntuación de los participantes salvados se debe mantener en el arreglo (no debe modificarse ni eliminarse).

Al finalizar el programa mostrar:

- El arreglo completo de puntuaciones.
- El participante eliminado y su puntuación.

PROGRAMACIÓN

Programador Universitario - Licenciatura en Informática - Ingeniería en Informática
Facultad de Ciencias Exactas y Tecnología - UNT

- La puntuación de cada participante salvado.

Utilice al menos las siguientes funciones:

```
void cargarArreglo(int arreglo[], int tama);  
void mostrarArreglo(int arreglo[], int tama);  
int buscarEliminado(int arreglo[], int tama, int salvado1, int salvado2, int salvado3);  
// devuelve el índice del participante eliminado
```

Desafío Opcional:

Si hay empate en la mínima nota, se debe **mostrar a todos los empatados** y dejar la decisión al coach líder. Actualice la función `buscarEliminado` por `buscarEliminadoConEmpate` de manera tal que:

- Busque el mínimo.
- Si hay empate.
 - Los guarde y muestre.
 - Pida el participante a eliminar (debe validar que el número ingresado corresponda a los empatados y no a otro).
- Finalmente, retorne el índice de participante eliminado.

Casos de Ejemplo

Ingrese el número de los 3 participantes salvados (1 a 8, sin repetir): 2 6 4 +++++ RESULTADOS FINALES ++++++ Puntuaciones de los participantes: Participante 1: 15 puntos Participante 2: 21 puntos Participante 3: 14 puntos Participante 4: 18 puntos Participante 5: 12 puntos Participante 6: 20 puntos Participante 7: 17 puntos Participante 8: 16 puntos Eliminado: Participante 5 con 12 puntos Puntuaciones de los participantes salvados: Participante 2: 21 puntos Participante 6: 20 puntos	Ingrese el número de los 3 participantes salvados (1 a 8, sin repetir): 2 3 6 Participantes con la menor puntuación (10 puntos): Participante 1 con 10 puntos Participante 4 con 10 puntos Participante 7 con 10 puntos Elija quien queda eliminado entre los empatados: 2 <i>Participante invalido, debe ser uno de los empatados.</i> Elija quien queda eliminado entre los empatados: 4 +++++ RESULTADOS FINALES ++++++ Puntuaciones de los participantes: Participante 1: 10 puntos Participante 2: 18 puntos
--	--

PROGRAMACIÓN

Programador Universitario - Licenciatura en Informática - Ingeniería en Informática
Facultad de Ciencias Exactas y Tecnología - UNT

Participante 4: 18 puntos	Participante 3: 22 puntos Participante 4: 10 puntos Participante 5: 17 puntos Participante 6: 19 puntos Participante 7: 10 puntos Participante 8: 20 puntos Eliminado: Participante 4 con 10 puntos Puntuaciones de los participantes salvados: Participante 2: 18 puntos Participante 3: 22 puntos Participante 6: 19 puntos
---------------------------	---

2. Suma de Matrices

Escriba un programa en **C** que permita al usuario:

1. Ingresar dos matrices cuadradas A y B de igual dimensión.
2. Calcular la **suma** de las matrices A+B.
3. Mostrar las tres matrices en pantalla: A, B y A+B.

Requisitos de implementación:

Desarrollar funciones para:

- **Cargar** los datos de una matriz. (Puede cargarla con valores aleatorios entre 0 y 9).
- **Mostrar** una matriz en pantalla.
- **Sumar** dos matrices y devolver el resultado en otra matriz.

Notas importantes:

- Una **matriz cuadrada** es aquella que tiene igual número de filas y columnas.
- La suma de dos matrices A y B de igual dimensión se obtiene sumando sus elementos correspondientes: $C[i][j] = A[i][j] + B[i][j]$

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} \quad A + B = \begin{pmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{pmatrix} = \begin{pmatrix} 6 & 8 \\ 10 & 12 \end{pmatrix}$$

Sugerencia: el programa puede solicitar al usuario la dimensión de las matrices y luego llamar a las funciones correspondientes para cargar, sumar y mostrar los resultados. Valide la dimensión ingresada por ejemplo entre 1 y 10.

PROGRAMACIÓN

Programador Universitario - Licenciatura en Informática - Ingeniería en Informática
Facultad de Ciencias Exactas y Tecnología - UNT

3. Adivinando la Palabra

Se desea programar un prototipo para un juego llamado *Adivinando la Palabra*. El funcionamiento del juego es el siguiente:

1. Ingreso de la palabra:

Un jugador ingresa la palabra que debe ser adivinada.

2. Preparación del juego:

El programa muestra guiones (“_”) que representan cada letra de la palabra ingresada, de acuerdo a su longitud.

3. Turnos del jugador adivinador:

- El jugador debe intentar adivinar la palabra ingresando una letra por turno.
- Si la letra ingresada está en la palabra, el programa actualiza los guiones, revelando todas las apariciones de esa letra.
- Si la letra no está en la palabra, se descuenta un intento.

4. Límites del juego:

- El jugador dispone de **3 intentos** para adivinar la palabra.
- El juego termina cuando el jugador adivina la palabra completa o agota sus intentos.

5. Resultados:

- Si el jugador adivina la palabra, el programa muestra un mensaje de felicitación.
- Si se agotan los intentos sin adivinar la palabra, el programa muestra un mensaje de derrota y revela la palabra correcta.

Ejemplo de ejecución:

Jugador 1 ingresa la palabra: **programacion**

Pantalla: _ _ _ _ _ _ _ _ _ _

Jugador 2 ingresa la letra: **a**

Pantalla: _ _ _ _ _ **a** _ **a** _ _ _ _ — *Intentos Disponibles: 3*

Jugador 2 ingresa la letra: **e** (No hay letra **e** en la palabra)

Pantalla: _ _ _ _ _ **a** _ **a** _ _ _ _ — *Intentos Disponibles: 2*